
1.1计算机发展历程
1.1.1计算机的硬件发展
发展变化
第一代： 电子管时代
特点
使用机器语言编程
容量小，体积大，成本高，性能低
第二代： 晶体管时代
特点
速度提升
软件使用高级语言
形成操作系统雏形
第三代： 中小规模集成电路时代
特点
使用半导体存储器
操作系统继续发展，出现分时操作系统
高级语言开始发展
第四代 ： 超大规模集成电路时代
特点
微处理器出现
并行，流水线，高速缓存，虚拟存储器出现
元件的更新变化
摩尔定律
18个月晶体管翻一倍
半导体存储器不断地发展
微处理器不断地发展
1.1.2计算机软件发展
面向机器
机器语言
汇编语言
面向问题的高级语言
FORTRAN--->PASCAL--->C++--->JAVA
1.1.3计算机的分类与发展方向
电子计算机
电子模拟计算机
电子数字计算机
数字计算机
专用计算机
通用计算机
根据效率、速度、价格、运行的经济性与适应性划分
通用计算机
巨型机、大型机、中型机、小型机、微型机、单片机
体积、功耗、性能、数据存储量、指令复杂程度、价格 -------> 依次递减
依照指令与数据流分类
单指令流与单数据流（SISD）
冯诺依曼体系结构
单指令流和多数据流（SIMD）
阵列存储器与向量存储器
多指令流和单数据流（MISD）
不存在
多指令流和多数据流（MIMD）
多处理器和多计算机系统
微型计算机：体积小 高性能 多用途
巨型机：更大 超高速 并行处理 更智能

1.2 计算机系统结构（上）

（数据库管理系统）DBMS与（数据库系统）DBS的区别

- DBMS是位于用户和操作系统之间的一层数据管理系统（系统软件）
- DBS是指计算机系统引入数据库后的系统
  - 组成结构
    - 数据库
    - 数据库管理系统
    - 数据库管理员
    - 应用系统

1.2.1计算机系统构成

- 硬件
- 软件

硬件与软件在逻辑上是等效的

- 功能实现
  - 1.功能使用频繁 2.硬件实现成本低　综上使用硬件实现
- 特点
  - 硬件实现效率要高于软件实现

1.2.2 计算机硬件的基本组成

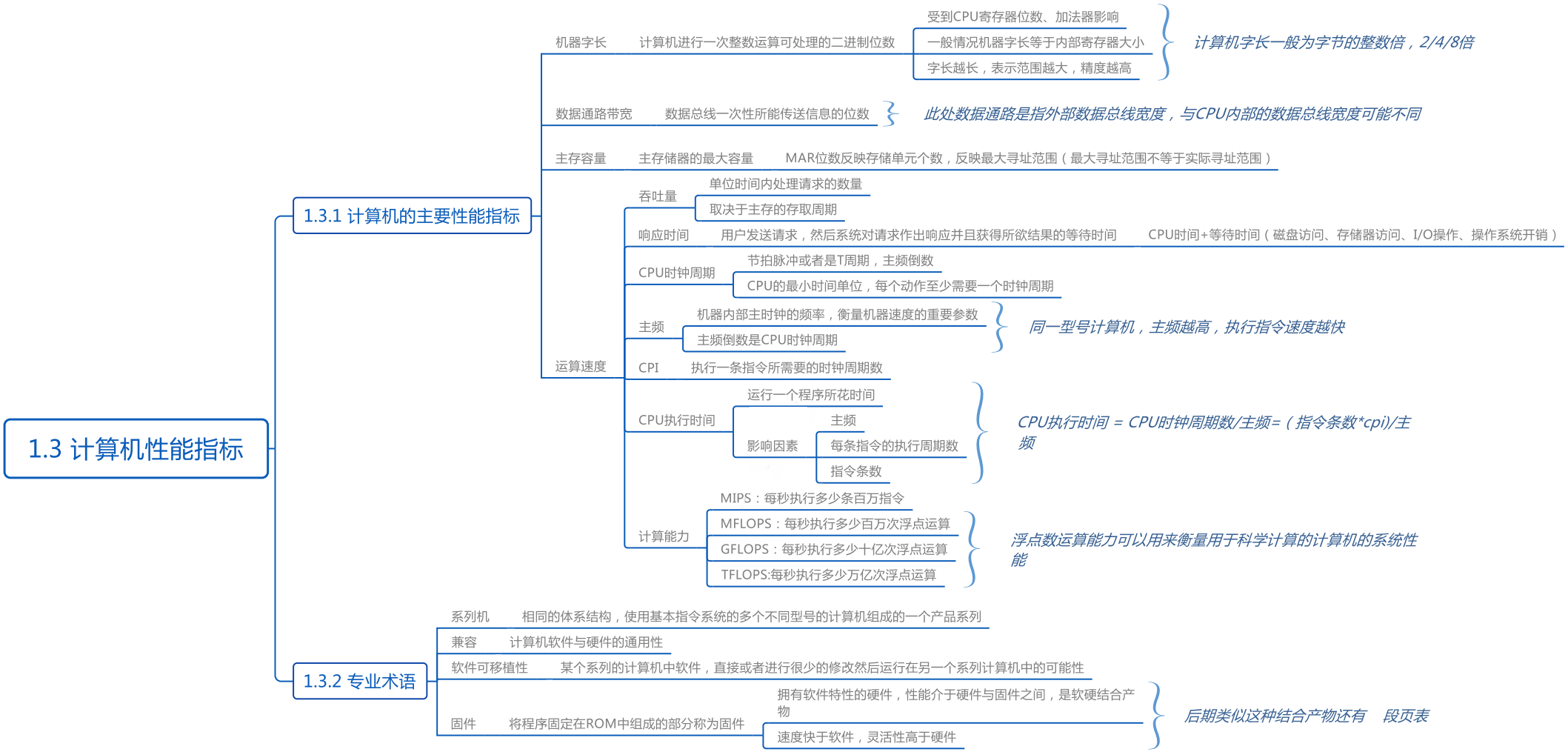
- 早期 冯诺依曼机
  - 组成： 运算器、存储器、控制器、输入设备、输出设备
  - 指令与数据共存于存储器，按照地址访问
  - 指令与数据以二进制代码构成
  - 指令组成：操作码与地址码
    - 操作码表示操作
    - 地址码表示操作数存储位置
  - 指令顺序存放，顺序执行（特定条件可改变执行顺序）
  - 运算器为中心，输入输出设备通过运算器与存储器传送数据（现代以存储器为中心）
  - “存储程序” ：将指令以二进制代码的形式事先输入计算机的主存储器，然后按其在存储器中的首地址执行程序的第一条指令，以后就按该程序的规定顺序执行其他指令，直至程序执行结束
- 现代计算机结构
  - 存储器为中心
  - I/O操作尽可能的绕开CPU，实现I/O设备与存储器直接完成，从而提高运行效率

1.2.3 计算机软件的分类

- 以功能进行分类
  - 系统软件
    - 操作系统（OS） 数据库管理系统（DBMS）、语言处理程序、分布式软件系统、网络软件系统、标准库程序、服务性程序等
  - 应用软件
    - 为用户解决某个问题的程序：科学计算类程序、工程设计类程序、数据统计与处理程序
- 语言分类
  - 机器语言（二进制代码语言） 机器语言是计算机唯一可以直接识别和执行的语言
  - 汇编语言　利用英文单词代替二进制指令代码，有助于记忆（必须要经过汇编程序对其进行翻译）
  - 方便程序设计人员使用的语言　高级语言——>汇编语言——>机器语言或者高级语言——>机器语言

最后一定会变成机器语言 因为机器语言唯一机器可以识别与执行的语言

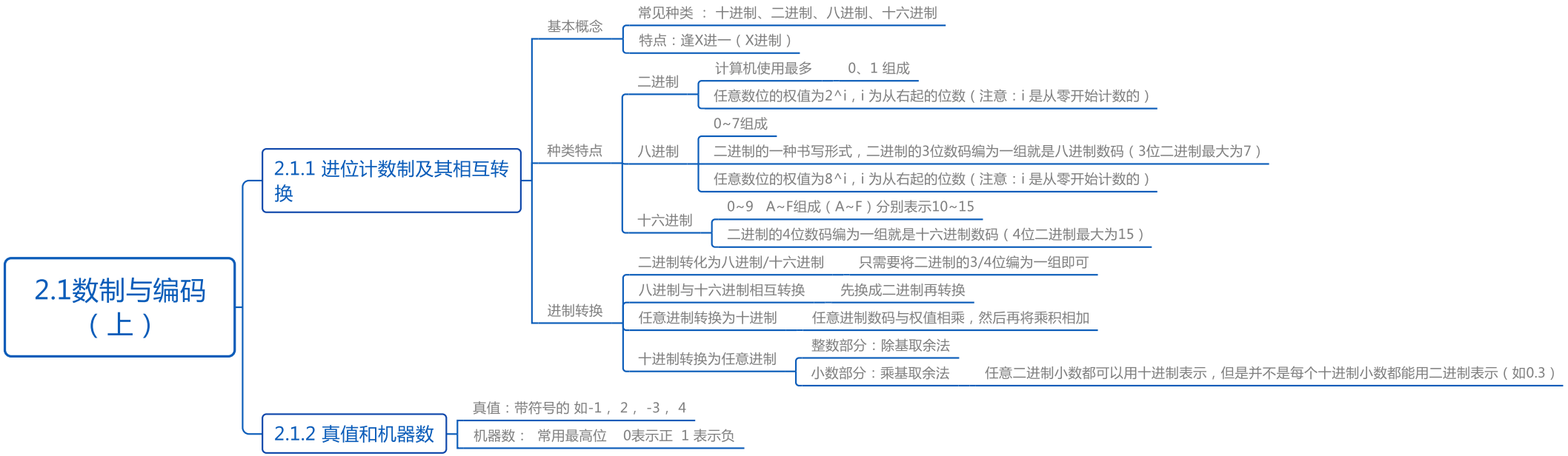

1.2计算机系统结构（下）
1.2.4计算机工作过程
具体步骤
程序、数据装入主存
从程序起始地址开始运行
程序首地址取出指令——>指令译码、指令执行——>完成功能并计算下一条指令地址
新得到的指令地址读出下一条指令，直到程序结束
信息流程
取指令 PC---->MAR---->M(存储器）---->MDR---->IR
分析指令 OP（IR）--->CU
执行指令 Ad（IR）--->MAR---->M----->MDR----->ACC
此外，取完指令PC+1，自动形成下一条指令地址
1.2.5计算机系统的的多级层次结构
虚拟机器（高级语言机器） 用汇编程序翻译成汇编语言程序
虚拟机器（汇编语言机器） 汇编程序翻译成机器语言程序
虚拟机器（操作系统机器） 机器语言解释操作系统
软硬件交互界面----------
传统机器（使用机器语言的机器） 微程序解释机器指令
微程序机器（微指令系统） 硬件直接执行微指令
各硬部件
主存
存储体 概念：存储元、存储单元、存储字、存储字长、地址
MAR 地址寄存器，用于指明要读/写哪个存储单元。其位数反映存储单元数量
MDR 数据寄存器，用于暂存要读/写的数据。其位数=存储字长
运算器
ACC 累加计数器，存放操作数、运算的结果
MQ 乘商寄存器，进行乘、除法时用得到
X 通用寄存器，存放操作数
ALU 算数逻辑单元，用电路实现各种算数运算、逻辑运算
控制器
PC 程序计数器，存放下一条指令的地址
IR 指令寄存器，存放当前执行的指令
CU 控制单元，分析指令，给出控制信号
工作过程
初始：指令、数据存入主存，PC指向第一条指令
从主存中取指令放入IR、PC自动加1、CU分析指令、CU指挥其他部件执行指令


1.3 计算机性能指标
1.3.1 计算机的主要性能指标
机器字长
计算机进行一次整数运算可处理的二进制位数
受到CPU寄存器位数、加法器影响
一般情况机器字长等于内部寄存器大小
字长越长，表示范围越大，精度越高
计算机字长一般为字节的整数倍，2/4/8倍
数据通路带宽
数据总线一次性所能传送信息的位数
此处数据通路是指外部数据总线宽度，与CPU内部的数据总线宽度可能不同
主存容量
主存储器的最大容量
MAR位数反映存储单元个数，反映最大寻址范围（最大寻址范围不等于实际寻址范围）
运算速度
吞吐量
单位时间内处理请求的数量
取决于主存的存取周期
响应时间
用户发送请求，然后系统对请求作出响应并且获得所欲结果的等待时间
CPU时间+等待时间（磁盘访问、存储器访问、I/O操作、操作系统开销）
CPU时钟周期
节拍脉冲或者是T周期，主频倒数
CPU的最小时间单位，每个动作至少需要一个时钟周期
主频
机器内部主时钟的频率，衡量机器速度的重要参数
主频倒数是CPU时钟周期
同一型号计算机，主频越高，执行指令速度越快
CPI
执行一条指令所需要的时钟周期数
CPU执行时间
运行一个程序所花时间
影响因素
主频
每条指令的执行周期数
指令条数
CPU执行时间 = CPU时钟周期数/主频=（指令条数*cpi)/主频
计算能力
MIPS：每秒执行多少条百万指令
MFLOPS：每秒执行多少百万次浮点运算
GFLOPS：每秒执行多少十亿次浮点运算
TFLOPS:每秒执行多少万亿次浮点运算
浮点数运算能力可以用来衡量用于科学计算的计算机的系统性能
1.3.2 专业术语
系列机
相同的体系结构，使用基本指令系统的多个不同型号的计算机组成的一个产品系列
兼容
计算机软件与硬件的通用性
软件可移植性
某个系列的计算机中软件，直接或者进行很少的修改然后运行在另一个系列计算机中的可能性
固件
将程序固定在ROM中组成的部分称为固件
拥有软件特性的硬件，性能介于硬件与固件之间，是软硬结合产物
速度快于软件，灵活性高于硬件
后期类似这种结合产物还有 段页表

传统冯诺依曼结构

- **输入设备**
  - 接收信息
  - 键盘、鼠标、扫描仪、摄像机
- **输出设备**
  - 将处理结果进行输出（让用户能看懂）
  - 显示器、打印机
- **存储器**
  - 主存
    - 功能或特点
      - CPU可以直接访问
      - 按地址存取信息　　相联存储器可根据内容访问（快表）
    - 结构：存储单元构成，一个存储单元所能存储的代码成为存储字，其长度就是存储字长
    - 组成：
      - 地址寄存器（MAR）：存放地址　　MAR位数与PC长度相等
      - 数据存储器（MDR）：主要暂存存储器中读或写的数据　　MDR位数与存储字长相等 一般为字节的二次幂的整数倍
      - 时序控制逻辑：　产生存储器操作的时序信号
  - 辅存　　帮助存储主存信息
- **运算器**
  - 对数据进行加工处理
  - 结构
    - 核心：　算数逻辑单元（ALU）
    - 累加器（ACC）、乘商寄存器（MQ）、操作数寄存器（X）、变址寄存器（IX）、基址寄存器（BR）等
    - 程序状态寄存器（PSW）保留运算指令和测试指令的结果状态，表征系统状态
- **控制器**
  - 计算机指挥中心
  - 组成结构
    - 程序计数器（PC）
      - 功能　　存放当前欲执行指令的地址，可以地址自动加一生成下一条指令地址
      - 结构　　与主存的MAR有着直接通路
    - 指令寄存器（IR）　　功能　　存放当前指令，内容来自MDR
    - 控制单元（CU）　　功能　　分析来自IR的数据，发出微操作命令

cpu：顾客　主存：商店　辅存：供货商　顾客购买商品，只能等到供货商将货物放入商店后，顾客才能购买


2.1数制与编码（上）
2.1.1 进位计数制及其相互转换
基本概念
常见种类 ： 十进制、二进制、八进制、十六进制
特点：逢X进一（X进制）
种类特点
二进制
计算机使用最多 0、1 组成
任意数位的权值为2^i，i 为从右起的位数（注意：i 是从零开始计数的）
八进制
0~7组成
二进制的一种书写形式，二进制的3位数码编为一组就是八进制数码（3位二进制最大为7）
任意数位的权值为8^i，i 为从右起的位数（注意：i 是从零开始计数的）
十六进制
0~9 A~F组成（A~F）分别表示10~15
二进制的4位数码编为一组就是十六进制数码（4位二进制最大为15）
进制转换
二进制转化为八进制/十六进制 只需要将二进制的3/4位编为一组即可
八进制与十六进制相互转换 先换成二进制再转换
任意进制转换为十进制 任意进制数码与权值相乘，然后再将乘积相加
十进制转换为任意进制
整数部分：除基取余法
小数部分：乘基取余法 任意二进制小数都可以用十进制表示，但是并不是每个十进制小数都能用二进制表示（如0.3）
2.1.2 真值和机器数
真值：带符号的 如-1， 2， -3， 4
机器数： 常用最高位 0表示正 1 表示负

2.1数制与编码（下）

2.1.3 BCD码

- 8421码（最常用）　注意：如果两个8421码相加后的和小于9（10进制），需要加6（10进制）进行修正
- 余3码（无权码）　在8421码的基础上加（0011），因为每个数都多3，所以称为余3码　8--->1011
- 2421码（有权码）
  - 权值由高到低为 2，4,2,1
  - >=5的4位二进制数中最高位为1 ，<5的最高位为0　5--->1011

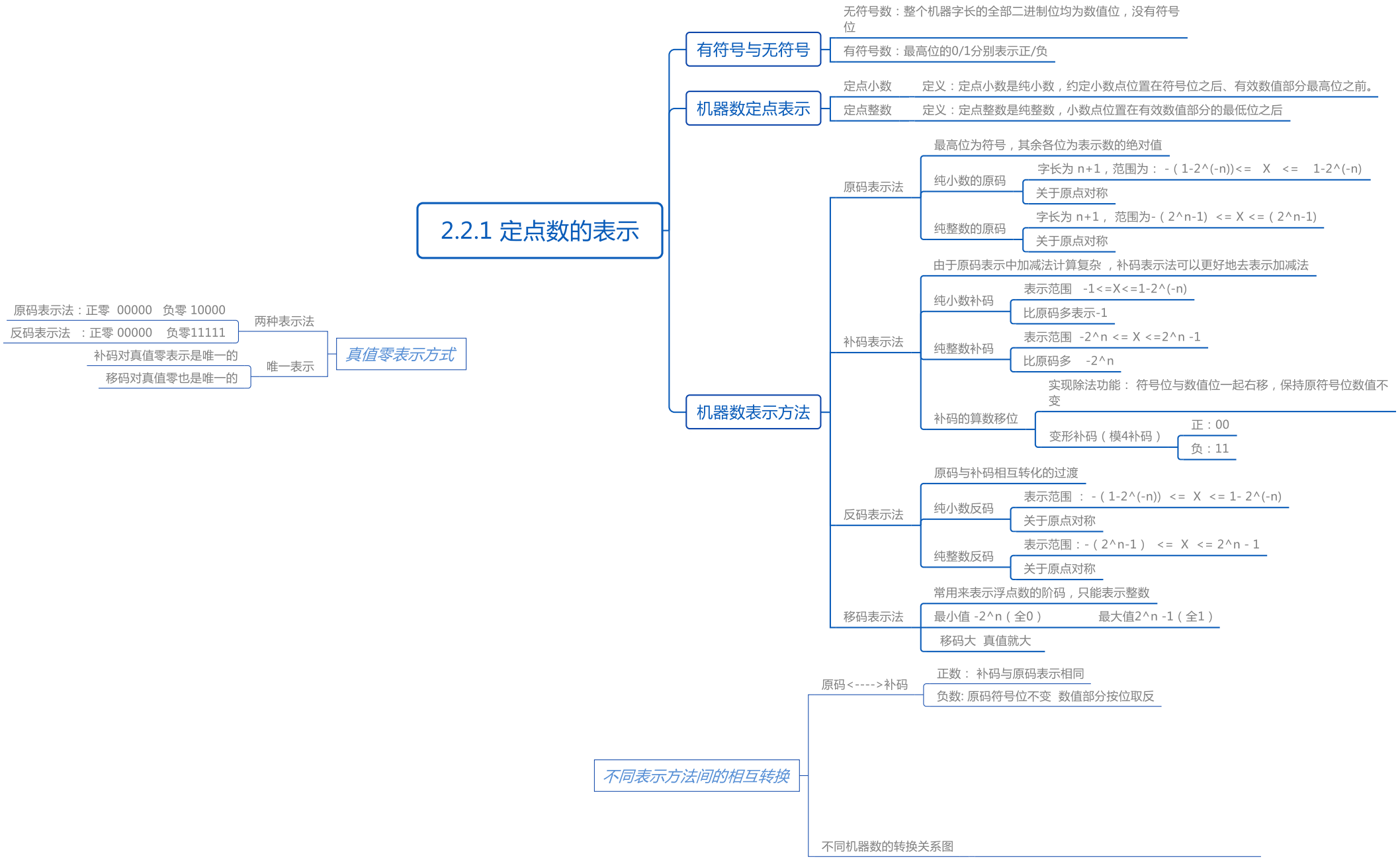
2.1.4 字符与字符串

- 字符串编码ASCII码　7位二进制编码
- 汉字的表示和编码
  - 每个编码用两个字节表示
  - 种类
    - 输入编码　计算机输入
    - 汉字内码　计算机内部处理
    - 汉字字形码　计算机输出
- 字符串存放
  - 小端模式　将数据的最高有效字节存放在高地址单元中
  - 大端模式　将数据的最高有效字节存放在低地址单元中
- 从低地址到高地址逐个字符存储，常采用'\0'作为结尾标志

2.1.5 校验码

- 概念：能够发现或者自动纠错的数据编码
- 原理：通过添加一些冗余码，实现检验或者纠错编码
- 分类
  - 奇偶校验码
    - 奇校验码：有效信息位和校验位中 1 的个数为奇数
    - 偶校验码：有效信息位和校验位中 1 的个数为偶数
    - 码距：2

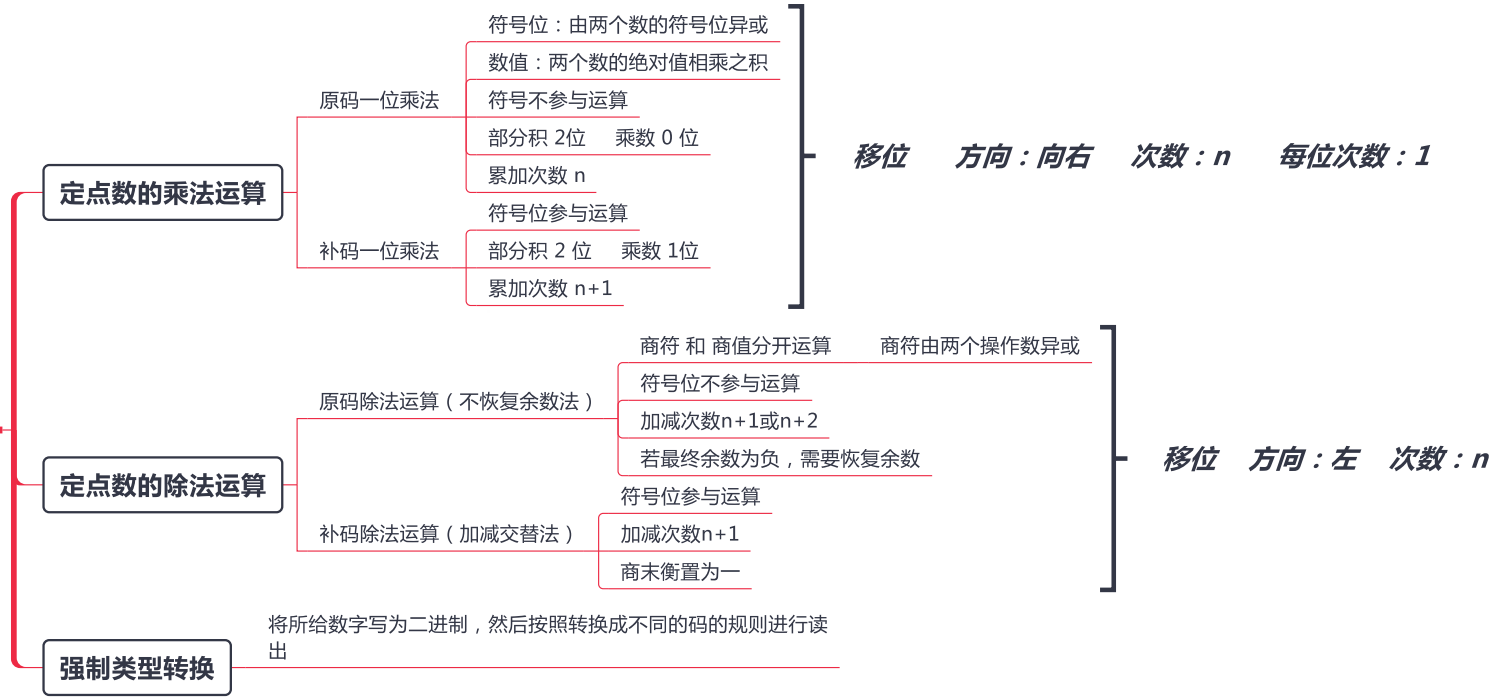
    } 但是如果编码中出现偶数位错误，无法检测
  - 海明码
    - 在有效信息位中添加几个校验码形成海明码
    - 不仅可以发现错位，还可以对错位进行纠错
    - 编码最小码距L越大，检测位数越多，纠错能力越强（纠错能力恒小于等于检测能力）　先检错，才能纠错
    - 补充
      - 海明码有1位纠错，2位检错能力
      - 为了区分1位错和2位错，还需添加"全校验位"对整体进行偶校验
      - 注意：有的题目位置编号可能是从小到大的，但处理方法雷同
  - CRC（循环冗余码）
    - 常用于大量的数据传送时的校验
    - 接收到循环冗余码后，对生成多项式做模2除法，余数为0则无错误
    - 余数不为0，对相应位置取反
    - 检错、纠错能力
      - 可检测出所有奇数个错误
      - 可检测出所有双比特的错误
      - 可检测出所有小于等于校验位长度的连续错误
      - 若选择合适的生成多项式，且2^R≥K+R+1，则可纠正单比特错


2.2.1 定点数的表示
有符号与无符号
无符号数：整个机器字长的全部二进制位均为数值位，没有符号位
有符号数：最高位的0/1分别表示正/负
机器数定点表示
定点小数
定义：定点小数是纯小数，约定小数点位置在符号位之后、有效数值部分最高位之前。
定点整数
定义：定点整数是纯整数，小数点位置在有效数值部分的最低位之后
机器数表示方法
原码表示法
最高位为符号，其余各位为表示数的绝对值
纯小数的原码
字长为 n+1，范围为：-(1-2^(-n))<= X <= 1-2^(-n)
关于原点对称
纯整数的原码
字长为 n+1， 范围为-(2^n-1) <= X <= (2^n-1)
关于原点对称
补码表示法
由于原码表示中加减法计算复杂 ，补码表示法可以更好地去表示加减法
纯小数补码
表示范围 -1<=X<=1-2^(-n)
比原码多表示-1
纯整数补码
表示范围 -2^n <= X <=2^n -1
比原码多 -2^n
补码的算数移位
实现除法功能： 符号位与数值位一起右移，保持原符号位数值不变
变形补码（模4补码）
正：00
负：11
反码表示法
原码与补码相互转化的过渡
纯小数反码
表示范围 ：-(1-2^(-n)) <= X <= 1- 2^(-n)
关于原点对称
纯整数反码
表示范围：-(2^n-1) <= X <= 2^n - 1
关于原点对称
移码表示法
常用来表示浮点数的阶码，只能表示整数
最小值 -2^n（全0） 最大值2^n -1（全1）
移码大 真值就大
真值零表示方式
两种表示法
原码表示法：正零 00000 负零 10000
反码表示法 ：正零 00000 负零11111
唯一表示
补码对真值零表示是唯一的
移码对真值零也是唯一的
不同表示方法间的相互转换
原码<---->补码
正数： 补码与原码表示相同
负数: 原码符号位不变 数值部分按位取反
不同机器数的转换关系图

2.2.2 定点数的运算（上）


定点数移位运算
算数移位
正数 移位后添零
负数
原码 添零
补码 左0右1
反码 全部填1
符号位不参与运算
逻辑移位
将操作数看做无符号数
左移或者右移 都要添零
循环移位
带进位标志位的循环移位
不带进位标志位的移位的循环移位
原码定点数的加减法运算
加法准则
符号相同：绝对值相加，符号不变
符号不同：绝对值大的减去绝对值小的 符号去绝对值大的数
减法准则 1.减数的符号取反 2.将其与被减数做原码加法运算
补码定点数加减法运算
1、参与的操作数均为补码
2、按照二进制规则运算 逢二进一
3、 符号位与数值位同时参与运算，符号位产生的进位丢掉，结果的符号由运算得出
4、补码运算的结果仍然是补码
符号扩展
正数 在原有的基础上，添零凑位数即可
负数
原码 将原有形式的符号移动到新形式的符号位上，新形式的附加位进行添零处理
补码 加1处理
反码 加1处理
溢出概念和判别方法
上溢：大于最大可以表示正数
下溢：小于最小可以表示的负数
补码判断溢出方法
一位符号位：参加运算的两个数的符号相同，但是结果符号出现变化，则结果溢出
双符号位：（模4补码）
00 正数 无溢出
01 正溢出
10 负溢出
11 负数 无溢出
一位符号位根据数据位进位判断 符号位进位与最高位进位相同，则无溢出

2.2.2 定点数的运算（下）


定点数的乘法运算
原码一位乘法
符号位：由两个数的符号位异或
数值：两个数的绝对值相乘之积
符号不参与运算
部分积 2位　乘数 0 位
累加次数 n
补码一位乘法
符号位参与运算
部分积 2 位　乘数 1位
累加次数 n+1
移位　方向：向右　次数：n　每位次数：1
定点数的除法运算
原码除法运算（不恢复余数法）
商符 和 商值分开运算　商符由两个操作数异或
符号位不参与运算
加减次数n+1或n+2
若最终余数为负，需要恢复余数
补码除法运算（加减交替法）
符号位参与运算
加减次数n+1
商末衡置为一
移位　方向：左　次数：n
强制类型转换
将所给数字写为二进制，然后按照转换成不同的码的规则进行读出

2.3.1浮点数的表示

- **表示格式**
  - 阶码是整数，阶符与阶码的位数共同反映浮点数的表示范围与小数点的实际位置
  - 数符表示浮点数的符号
  - 尾数的位数表示浮点数的精度
- **规格化浮点数**
  - 左规：尾数算数左移一位，阶码减一
  - 右规：结果出现溢出（双符号位01或10），将尾数算数右移一位，阶码加1

  左规可以多次，右规只能一次
- **原码规格化**
  - 正数：0.1XXX
  - 负数： 1.1XXX

  基数：4　原码规格化形式的尾数最高2位不全为0
  基数：8　原码规格化形式尾数最高3位不全为0
- **补码规格化**
  - 正数：0.1XXX
  - 负数：1.0XXXX
- **IEEE754**
  - 整数部分隐含1
    - 短浮点数（组成位数）
      - 数符：1
      - 阶码 8
      - 尾数数值：23
      - 总位数：32

      偏置值：7fh（16进制）　127（十进制）
    - 长浮点数（组成位数）
      - 数符：1
      - 阶码 11
      - 尾数数值：52
      - 总位数：64

      偏置值：3ffh（16进制）　1023（十进制）
  - 临时浮点数（组成位数）
    - 数符：1
    - 阶码 15
    - 尾数数值：64
    - 总位数：80

    偏置值：3fffh（16进制）　16383（十进制）
  - 浮点数范围
    - 单精度浮点数：2^（-126）~2^(127)*(2-2^(-23))
    - 双精度浮点数：2^(-1022)~2^(1023)*(2-2^(-52))
- **定点数与浮点数的区别**
  - 相同字长，浮点数范围更大
  - 浮点数精度降低
  - 浮点数运算复杂
  - 浮点数只有规格化后阶码超出表示范围，才发生溢出

2.3.2 浮点数的加减运算

- **运算步骤**
  - 对阶：小阶看齐大阶，阶码小的尾数右移一位，阶加一，直到阶码相等
  - 尾数求和　尾数按照定点数加减规则运算
  - 规格化
    - 最高数值位与符号位不同即为规格化形式
    - 左规：尾数左移1位，和的阶码减1　直到00.1XX或者11.0XXX
    - 右规，尾数求和结果溢出（10.XXX或者01.XXX） 尾数右移一位，和的阶码加1
  - 舍入
    - 0舍1入法
      - 尾数右移时，移去的最高数值位为0，则舍去
      - 尾数左移时，移去的最高数值位为1，则尾数末位加1
    - 恒置1法　尾数右移，不论最高数值位丢掉的是1还是0，都将尾数末位恒置为1

    可能会导致溢出，此时需要再一次右规
  - 溢出判断
    - 只有右规后，仍然溢出， 此时才是真正溢出
    - 上溢出：进入中断处理
    - 下溢出：按机器零处理
  - 强制类型转换
    - char-->int　在前面补0
    - int <----->unsigned　彼此都可能因为溢出丢失数据
    - int<------>float
      - float转换为int可能会出现精度损失和溢出
      - int转换为float 可能会出现数据舍入
- **边界对齐**
  - 现代计算机通常是按字节编址，即每个字节对应1个地址
  - 通常也支持按字、按半字、按字节寻址

2.4算术逻辑单元（ALU）

- **运算器组成：算术逻辑单元 累加器 状态寄存器 通用寄存器组**
- **2.4.1串行加法器和并行加法器**
  - 一位全加器
    - 两个加数输入，以及低位进入输入
    - 本位结果和进位输出
  - 串行加法器
    - 只有一个全加器，数据逐位的送入加法器中运算，逐位送回寄存器
    - 操作数n位 则进行n次
    - 成本低，但是速度慢
  - 并行加法器
    - 多个加法器共同组成，每个全加器都有一个低位送来的进位输入，向高位的进位输出
    - 进位方式
      - 串行进位 将全加器串接在一起，每级进位依赖于前一级进位
      - 并行进位 同时进位，各级进位信号同时形成

提高并行加法器速度的关键在于加快进位产生和传递速度

- **2.4.2 算术逻辑单元的功能和结构**
  - ALU：算数运算与逻辑运算

虚拟存储器和CACHE异同

- **相同之处**
  - 目的为了提高系统性能
  - 数据分为小信息块，作为基本的传递单位
  - 都存在地址映射，替换算法，更新策略
  - 按照局部性原理，将活跃的数据放到高速部件中
- **不同之处**
  - Cache解决系统速度问题　苏尼存储器解决主存容量问题
  - Cache 全部由硬件实现，是硬件存储器 虚拟存储器有OS和硬件共同实现，是逻辑上的存储器
  - Cache对所有程序员透明，虚拟存储器对应用程序员透明，对系统程序员不透明
  - 虚拟存储器不命中对系统性能影响更大
  - CPU只能与Cache和主存直接交互　虚拟存储系统只能先将数据从硬盘调入主存，不能与CPU直接通信


3.1 ~3.2存储器的层次结构
3.1.1存储器分类
层次分类
主存
存放计算机运行时的程序与数据
可以被CPU直接访问，也可以与Cache和辅存进行交换数据
容量小，速度快，价格高
辅存
存储当前暂时不用的程序和数据，以及一些永久保留数据
不可以与CPU直接交互
容量大，速度慢，价格低
高速缓冲存储器（cache）
位于主存与CPU之间，缓解CPU与主存之间速度差距问题
CPU可以直接访问
容量小，速度与CPU相匹配，价格高 高档计算机将他们放在CPU中
存储介质分类
磁表面存储器：磁盘 磁带
磁芯存储器半导体存储器（MOS型存储器 双极型存储器）
光存储器：光盘
存取方式分类
随机存储器（RAM）
随机存取，读写方便，使用灵活
用途：主存 高速缓冲存储器
类型： 静态RAM(触发器原理） 动态RAM（电容充电原理）
只读存储器（ROM）
只能读出不能写入
用途： 与随机存储器构成主存 存储固定不变的程序（甚至操作系统的固化）
ROM也派生出可以反复重写的类型
rom与ram都是随机存取，广义上的只读存储器可以通过电擦除写入的，写入速度比读速度慢
信息可保存性
易失性存储器
RAM
断电后，仍然可以保持信息
非易失性存储器存储器
ROM、磁表面存储器、光存储器
读出类型
破坏性读出：读出后需要对存储信息再生
非破坏性读出：不会对原存储信息破坏
3.1.2 存储器的性能指标
计算机追求目标：大容量，低成本，高速度（联系自身买电脑）
存储容量=存储字数*字长
单位成本=总成本/总容量
存储速度： 数据传输率 = 数据宽度/存储周期
存取时间：存储器启动到完成操作，分为读出时间与写入时间
存取周期（读写周期、访问周期）：存储器进行一次完整读写操作需要时间
主存带宽：数据传输率，每秒主存进出信息的最大数量
存储周期一般大于存取时间（读写操作后需要内部复原时间）
3.2 存储器的层次化结构
存储系统目标：更大更快更便宜
Cache-----主存层次 解决CPU和主存速度不匹配问题
主存-------辅存层次 解决存储系统容量问题
主存----辅存层次不断发展，衍生出虚拟存储器系统

3.3 半导体随机存储器（上）

3.3.1 半导体存储芯片

- 存储矩阵：大量相同的位存储单元阵列构成
- 译码驱动：地址信号翻译成对应存储单元的选通信号
- 读写电路：完成读写操作
- 读/写控制线：决定芯片是读还是写
- 片选线：确定那个芯片被选中
- 地址线：单项输入，位数与存储字的个数有关
- 数据线：双向的，位数与读出或写入的数据位数有关

数据线数与地址线数共同反映存储芯片容量大小

- 半导体随机存储器分类（存储原理不同）
  - SRAM：高速缓存
  - DRAM：主存
- 74138译码器

3.3.2 SRAM和DRAM

- SRAM
  - 使用双稳态触发器（六管MOS）记忆信息
  - 非破坏性读出，易失性存储器
  - 存取速度快，集成度低，功耗大，成本高,常用来组成高速缓冲存储器
  - 同时送行列地址
- DRAM
  - 利用电荷存储信息
  - 破坏性读出，易失性存储器
  - 存取速度慢，集成度高，功耗低，容量大 ，成本低， 常用来组成主存系统
  - 分两次送行列地址
  - 刷新方法（DRAM特有）
    - 刷新单位是行
    - 集中刷新
      - 刷新时间固定 存在死区
      - 读写不受刷新影响，存取速度快
      - 死区不能访问存储器
    - 分散刷新
      - 将每行刷新分散到各个工作周期中
      - 没有死区
      - 存取周期边长，降低整机速度
    - 异步刷新
      - 集中刷新和异步刷新结合
      - 缩短了死时间，又提高了整机速度
    - 透明刷新
      - 刷新安排在译码阶段，不存在死时间

死区：在刷新的时候，停止对存储器的读写操作，称为死时间

- 存储器的读写周期
  - RAM读周期：存储芯片进行两次连续读操作时，必须间隔的时间，读周期总是大于等于读出时间
  - RAM写周期：数据总线上的信息能够可靠的写入存储器

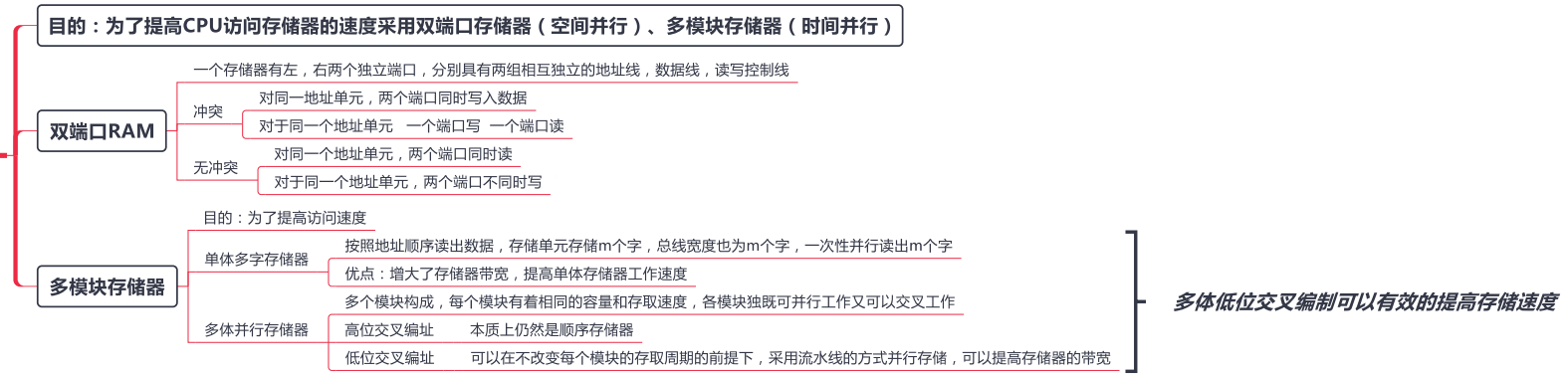
3.3.3只读存储器

- ROM特点
  - 随机存取，非易失性存储器
  - 结构简单，位密度比可读写存储器高
- ROM类型
  - 掩膜式只读存储器（MROM）
    - 可靠性高，集成度高，价格便宜
    - 灵活性差
  - 一次性可编程只读存储器（PROM）
    - 写入内容无法更改
  - 可擦除可编程只读存储器（EPROM）
    - 可以对内容进行多次改写
    - 紫外线擦除UVEPROM
    - 电擦除E^2PROM
  - 闪速存储器（Flash Memory)
    - 可长期保存信息 可重写
    - 价格便宜，集成度高
    - 擦写速度快
  - 固态硬盘(SSD)
    - 可以长期保存信息，快速擦除，重写
    - 相对于传统硬盘，读写速度快，低功耗
    - 价格高

3.4 主存储器和CPU的连接



3.5 双端口RAM和多模块存储器



3.6高速缓冲存储器

程序访问的局部性原理

- 时间局部性：将要使用的信息，可能是现在正在使用的信息
- 空间局部性：将来使用的信息，可能在正在使用信息的存储空间的附近
- 基于局部性原理创造出高速缓冲技术（Cache）

CACHE基本工作原理

- 通常使用SRAM制造
- 存储主存中最为活跃的信息副本，按照某种策略将这些活跃的信息存入到Cache中
- CPU发出读请求
  - Cache命中：直接对Cache进行读操作
  - Cache不命中：CPU访问主存操作，并且将访问数据送入到Cache中

cpu与cache之间交换数据的基本单位是 字
cache与主存之间交换数据的基本单位是 cache块
注意：某些计算机也可能是同时访问cache和主存

CACHE与主存的映射方式

- 按照一定的规则将主存中的某些数据存入到Cache中
- 直接映射
  - 主存数据块只能装入Cache中的唯一位置
  - 地址结构　主存字块标记　Cache字块地址　字块内地址

冲突率高，利用率低，实现简单

- 全相联映射
  - 主存数据库可以放在Cache中的任何位置
  - 地址结构　主存字块标记　字块内地址

地址变换慢，实现成本高　比较灵活 冲突率低

- 组相邻映射
  - 将Cache分为不同的组，主存的数据库可以装入一组内的任何位置
  - 地址结构　主存字块标记　组地址　字块内地址

CACHE中的替换算法

- 随机算法：随机确定替换的Cache块
  - 优点：实现简单
  - 缺点：没有依据局部性原理，命中率低
- 先进先出算法（FIFO）：最早调入的行进行替换
  - 优点：容易实现
  - 缺点:没有依据局部性原理
- 近期最少使用算法（LRU）:根据局部性原理，选择近期内最久没有访问的存储行
  - 优点：平均命中率高的
  - 缺点：需要设置计数器比较存储行

堆栈类算法

- 最不经常使用算法：一段时间内访问次数最少的存储行换出

CACHE写策略

- 写命中
  - 全写法（写直通法、write-through）:对Cache写命中后，数据同时写入Cache 和 主存
    - 实现简单，随时保持主存数据正确性
    - 增加了访存次数，降低了效率
  - 写回法（write-back）：对Cache命中时，只修改Cache内容，不立即写入主存
    - 减少了访存次数
    - 存在数据不一致的隐患，同时需要设置一个脏位
- 写不命中
  - 写分配法（write-allocate):加载主存中的块到Cache中，然后更新Cache块
    - 试图使用空间局部性原理
    - 每次不命中都要从主存中调块
  - 非写分配法（not-write-allocate):只写入主存，不进行调块
- 多级Cache（通常为3级）　可以有效避免频繁写时造成的写缓冲饱和和溢出

非写分配法与全写法搭配
写分配法与写回法搭配

3.7虚拟存储器

基本概念

- 将主存或者辅存的地址空间统一编址
- 实地址对应的是主存地址空间
- 使用虚地址需要辅助硬件找出虚地址和实地址之间的关系，并对其对应存储单元装入状态进行判断
- 实际情况：需要使用的先送入主存，暂时不用的放在磁盘中

页式虚拟存储器

- 以页为基本单位的虚拟存储器称为页式虚拟存储器
- 计算过程
  - 虚拟地址 = 虚页号+页内地址
  - 虚页号+页表起始地址 = 页表地址　　根据页表地址查找实页号
  - 实页号+页内地址 = 实地址
  - 子主题 4

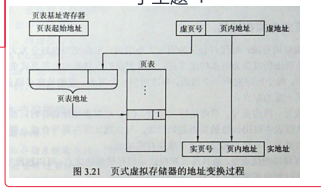


图 3.21　页式虚拟存储器的地址变换过程

优点：页面长度固定 调入方便
缺点：零头浪费 对页的处理、保护、共享不是特别方便

段式虚拟存储器

- 按照程序的逻辑结构划分
- 计算过程
  - 虚地址 = 段号 + 段内地址
  - 段号+段表起始地址 = 段表地址　　查询段表数据
  - 实地址 = 段表所得数据+ 段内地址

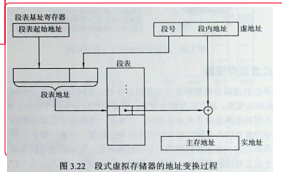


图 3.22　段式虚拟存储器的地址变换过程

优点： 段分界与程序分界相对应 易于编译、管理、修改、保护、共享
缺点：段长分配不便 存在段间碎片

段页式存储器

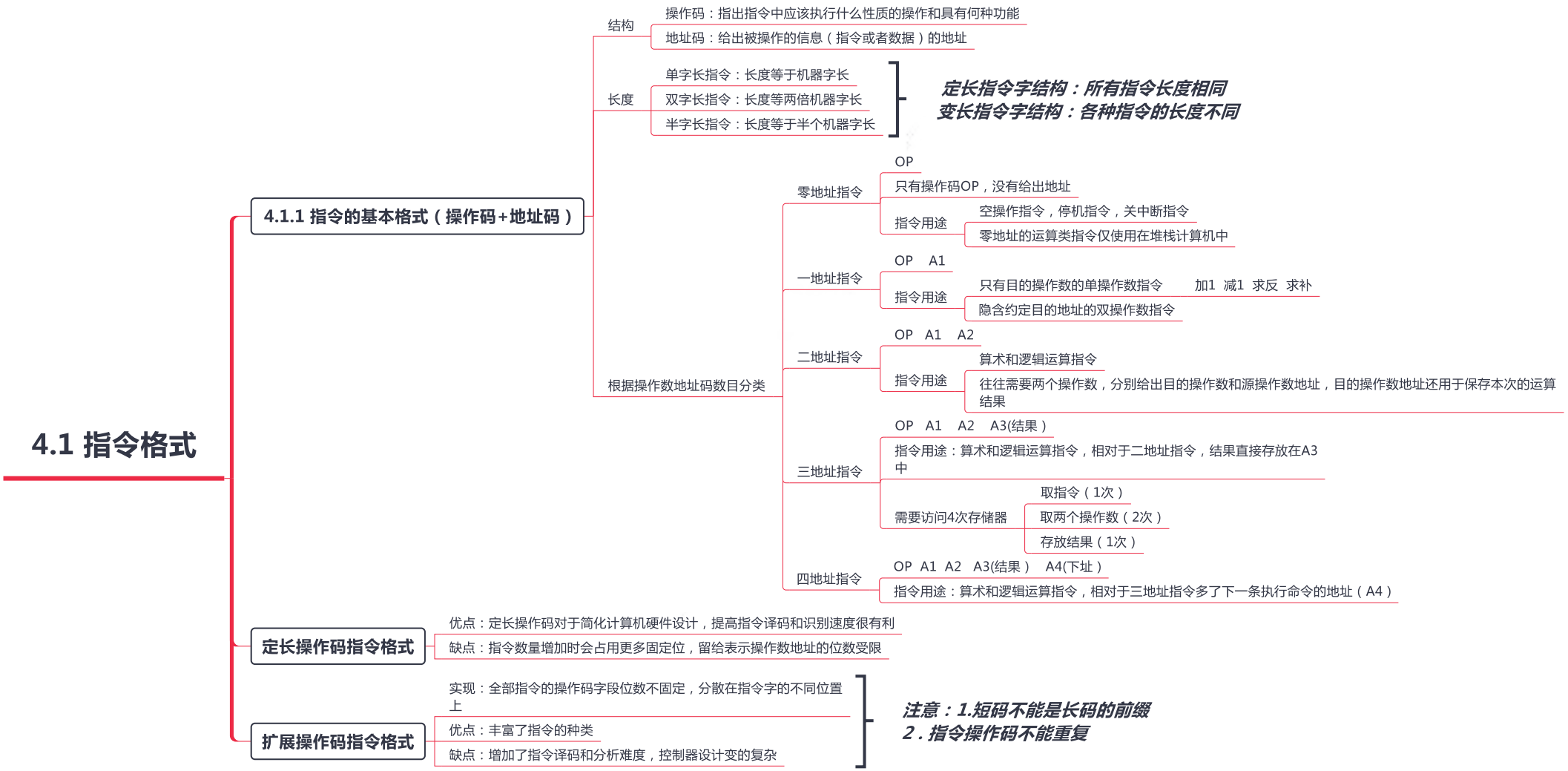
- 先将程序按逻辑分段，再将每段分为固定大小页
- 段长必须是页的整数倍
- 虚地址 = 段号 + 段内页号 + 页内地址

优点： 可以按段实现共享和保护，同时也有着页的调用方便
缺点：地址变换要两次查表，开销较大

快表（TLB）

- 根据局部性原理，将一些经常访问的页放入高速缓冲器中构成快表，可以极大提高查询的效率
- 采用相联存储器构成，可以按照内容查询
- 访问顺序：TLB--->页表--->Cache--->主存
- 命中情况
  - Cache命中，page必然命中，TLB不一定命中
  - Cache不命中，无法退出TLB与page命中情况
  - Page不命中，Cache和主存不会命中，此时要执行调页策略

只要抓住存储数据的来源就可以推导出命中情况


4.1 指令格式
4.1.1 指令的基本格式（操作码+地址码）
结构
操作码：指出指令中应该执行什么性质的操作和具有何种功能
地址码：给出被操作的信息（指令或者数据）的地址
长度
单字长指令：长度等于机器字长
双字长指令：长度等两倍机器字长
半字长指令：长度等于半个机器字长
定长指令字结构：所有指令长度相同
变长指令字结构：各种指令的长度不同
根据操作数地址码数目分类
零地址指令
OP
只有操作码OP，没有给出地址
指令用途
空操作指令，停机指令，关中断指令
零地址的运算类指令仅使用在堆栈计算机中
一地址指令
OP A1
指令用途
只有目的操作数的单操作数指令 加1 减1 求反 求补
隐含约定目的地址的双操作数指令
二地址指令
OP A1 A2
指令用途
算术和逻辑运算指令
往往需要两个操作数，分别给出目的操作数和源操作数地址，目的操作数地址还用于保存本次的运算结果
三地址指令
OP A1 A2 A3(结果）
指令用途：算术和逻辑运算指令，相对于二地址指令，结果直接存放在A3中
需要访问4次存储器
取指令（1次）
取两个操作数（2次）
存放结果（1次）
四地址指令
OP A1 A2 A3(结果） A4(下址）
指令用途：算术和逻辑运算指令，相对于三地址指令多了下一条执行命令的地址（A4）
定长操作码指令格式
优点：定长操作码对于简化计算机硬件设计，提高指令译码和识别速度很有利
缺点：指令数量增加时会占用更多固定位，留给表示操作数地址的位数受限
扩展操作码指令格式
实现：全部指令的操作码字段位数不固定，分散在指令字的不同位置上
优点：丰富了指令的种类
缺点：增加了指令译码和分析难度，控制器设计变的复杂
注意：1.短码不能是长码的前缀
2．指令操作码不能重复

4.2 指令寻址方式


4.2.1 指令寻址和数据寻址
指令寻址
顺序寻址：通过程序计数器（PC）加1，自动生成下一条指令的地址
跳跃寻址
通过转移类指令实现，是否跳跃受到状态寄存器和操作数的控制
跳跃到的地址分类
绝对地址：由标记符直接得到
相对地址：相对于当前指令地址的偏移量
跳跃的结果是当前指令修改pc值
数据寻址
在指令中表示一个操作数的地址
结构： 操作码+寻址特征+形式地址A
4.2.2 常见的数据寻址方式
隐含寻址
不显示的给出操作数地址，在指令中隐含操作数地址
优点：有利于缩短指令字长
缺点：增加存储操作数或隐含地址的硬件
不访存
立即（数）寻址
地址字段给出的不是操作数的地址，而是操作数本身，又称为立即数
优点：不需要访问主存，指令执行时间最短。
缺点：位数限制了立即数的范围
不访存
直接寻址
指令字中的 形式地址就是真实地址
优点：访问一次主存，不需要专门计算操作数的地址
缺点：形式地址的位数限制了寻址范围，操作数地址不易修改
访存1次
间接寻址
指令的地址字段给出存储地址信息的地址（类似于套娃）
优点：扩大了寻址范围，方便编制程序
缺点：需要多次访存，访问速度过慢，这种寻址方式不是特别常用
一般问到扩大寻址范围，通常指寄存器间接寻址 访存多次（具体看几次间接寻址）
寄存器寻址
指令字中直接给出操作数所在的寄存器编号
优点：不访问主存，执行速度快，支持向量/矩阵运算
缺点：寄存器价格高昂，寄存器中的寄存器个数有限
不访存

王道论坛
专注于计算机考研的点点滴滴

4.3 CISC 和 RISC概念



5.1CPU的功能和基本结构



5.2 指令执行过程

指令周期

- 概念
  - CPU从主存中每取出一条指令所需要的全部时间称为指令周期，即CPU完成一条指令的时间
  - 一个指令周期包含多个机器周期，一个机器周期包含若干个时钟周期，是CPU操作的基本单位
  - 每个指令周期内的机器周期数可以不等，每个机器周期内的节拍数可以不等
- 基本组成
  - 取指周期：取出指令
  - 间址周期：取出有效地址
  - 执行周期：取出操作数
  - 中断周期：保存程序断点
- 不同指令的指令周期
  - 无条件转移指令，不需要访存，所以只有取指周期、执行周期
  - 对于间接寻址的指令，首先要进行访存，取出有效地址，根据有效地址取出操作数，所以其具有间址周期
  - 当CPU采用中断方式实现主机和I/O设备的信息交换的时候，CPU在每次指令执行结束前都要发出中断查询信号，所以需要中断周期

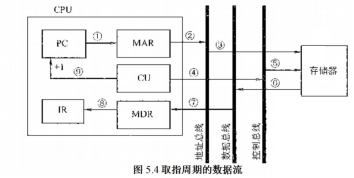


图 5.4 取指周期的数据流

指令周期的数据流

- 取指周期
  - CU发出控制信号--->控制总线----->主存
  - 主存--->数据总线---->MDR---->IR(存放指令）
  - CU发出读命令-->PC内容加1

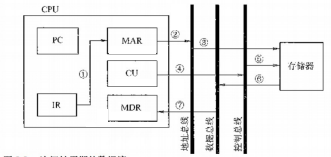


图 5.5 一次间址周期的数据流

- 间址周期
  - Ad（IR）（或MDR） ---->MAR---->地址总线---->主存
  - CU发出读命令--->控制总线--->主存
  - 主存---->数据总线--->MDR （存放有效地址）
- 执行周期
  - 执行周期根据IR中的数据和ALU操作的产生执行结果，不同指令的执行过程不同

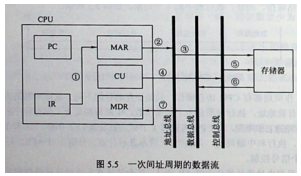


图 5.5 一次间址周期的数据流

- 中断周期
  - CU控制将SP减1, SP--->MAR--->地址总线--->主存
  - CU发出写命令--->控制总线--->主存
  - PC--->MDR--->数据总线--->主存（程序断点存入主存）
  - CU （中断服务程序的入口地址）---->PC

指令执行方案

- 单指令周期
  - 所有指令完成时间相同
  - 指令串行执行，指令周期取决于最长指令执行时间
  - 降低了系统整体运行速度
- 多指令周期
  - 不同类型的指令选用不同的执行步骤完成
  - 指令串行执行，对于不同指令分配不同的周期
- 流水线方案
  - 指令之间并行，将每个时钟周期都利用起来
  - 尽量让多个指令并行运行

cisc可以通过优化实现流水线　　risc必须要实现流水线

5.3 数据通路的功能和基本结构

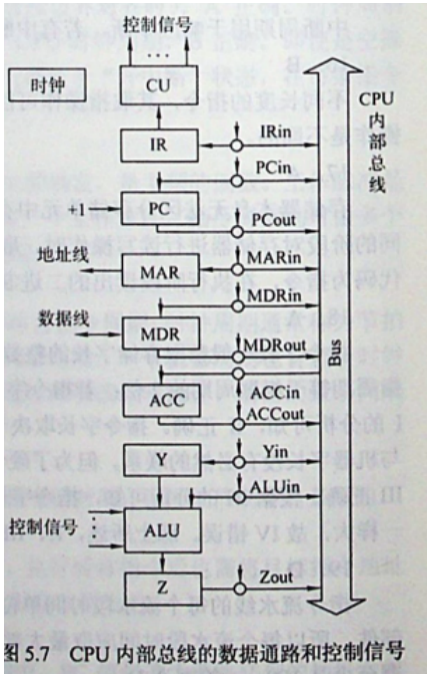


图 5.7 CPU 内部总线的数据通路和控制信号

- **数据通路的功能**
  - 通路结构图
  - 概念： 数据在功能部件之间传送的路径
  - 功能：实现CPU内部的运算器与寄存器之间的数据交换
- **数据通路的基本结构**
  - 基本机构分类
    - CPU内部单总线方式
      - 将所有寄存器的输入输出端连接在同一条公共通路上
      - 优点：结构简单
      - 缺点：存在冲突现象，性能较低
    - CPU内部三总线方式
      - 将所有寄存器的输入输出端连接在多个公共通路上
      - 优点：执行效率高
      - 缺点：实现复杂
    - 专用数据通路方式
      - 减少使用共享线路，专线专用
      - 优点：性能高
      - 缺点：实现复杂，硬件量大
  - 数据传送
    - 寄存器之间的数据传送：寄存器之间的数据传送可以通过CPU内部总线完成
    - 主存与CPU之间的数据传送：主存与CPU 之间的数据传送也需要借助CPU内部总线完成
    - 执行算数或逻辑运算：由于ALU本身没有内部存储功能的组合电路，所以相加的操作数，必须在ALU两端同时有效

实现简单，往往会伴随有性能低下或者其他问题

性能的提高，往往会伴随着复杂的实现

5.4 控制器的功能和工作原理（上）

- **控制器的结构和功能**
  - 结构
    - 运算器部件通过数据总线与内存储器、输入设备和输出设备传送数据
    - 输入设备和输出设备通过接口电路与总线相连接
    - 内存储器、输入设备和输出设备从地址总线接收地址信息，控制总线得控制信号，数据总线与其他部件传送数据
  - 功能
    - 主存中取出指令，产生下一条指令在主存中的地址
    - 对指令进行译码或者测试，产生相应的操作控制信号，以便启动规定的动作
    - 指挥并控制CPU、主存、输入和输出设备之间的数据流动方向
- **根据产生微操作信号的方式不同，存在硬布线控制器和微程序控制器**
- **硬布线控制器**
  - 根据指令要求、当前的时序以及外部和内部的状态，按照时间的顺序发送一些微操作控制信号　　又称为组合逻辑控制器
  - 控制单元（CU）信号来源
    - 指令译码器产生的信息
    - 时序系统产生的机器周期信号和节拍信号
    - 来自执行单元的反馈标志

    注意：控制单元还接收来自系统总线的控制信号：中断请求　dma请求
  - 硬布线控制器的时序系统及微操作
    - 时钟周期：用时钟信号控制节拍发生器：每个节拍内机器可以完成或几个需要同时执行的操作
    - 机器周期：机器周期可视为所有指令执行过程中的一个基准时间
      - 通常我们以存取周期作为基准时间，即从内存中读取一个指令字的最短时间作为机器周期
      - 在存储字长等于指令字长的前提下，取指周期可以作为机器周期
    - 指令周期
      - 取指周期，间址周期，执行周期，中断周期
      - 见导图5.2 指令执行过程
    - 微操作命令分析　　控制单元发出各种操作序列，这些命令必须要按照一定的次序才能使得机器有序的工作
  - CPU控制方式
    - 同步控制
      - 使用统一的时钟
      - 优点：控制电路简单
      - 缺点：运行速度慢
    - 异步控制
      - 不存在基准时标信号，每个部件按照自身的速度工作
      - 优点：速度快
      - 缺点：电路复杂
    - 联合控制
      - 同步和异步结合
      - 大部分部件同步，小部分由异步
  - 硬布线控制器设计
    - 列出微操作命令时间表
    - 对微操作信号综合
    - 画出微操作命令的逻辑图

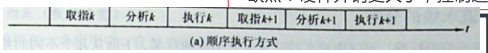
5.4 控制器的功能和工作原理（下）

- 微程序控制器
  - 基本概念
    - 采用存储逻辑实现，将微操作信号代码化，控制存储器存储微程序，微操作控制信号由微指令产生
    - 微命令与微操作
      - 机器指令分解----》微操作序列　　控制部件向执行部件发出的控制命令称为微命令，是构成控制序列的最小单位
      - 微命令具有相容和相斥性
      - 操作控制字段：用于产生某一步操作需要的各种操作控制信号
      - 顺序控制字段：控制产生下一条要执行的微指令地址
    - 主存储器与控制存储器
      - 主存储器：存储程序和数据，RAM组成
      - 控制存储器：存放微程序，ROM组成
    - 程序和微程序
      - 程序：指令的有序集合
      - 微程序，微命令的有序集合
      - ***一条指令的功能由一段微程序来实现***
    - 微地址寄存器（CMAR）：用于存放控制存储器读/写微指令的地址
    - 微指令寄存器（CMDR或者μIR）：用于存放从控制存储器中读出的微指令
  - 微程序控制器组成和工作过程
    - 基本组成
      - 控制存储器：存放指令对应的微程序，ROM构成
      - 微指令寄存器：存放微指令
      - 微地址形成部件：产生初始微地址和后继微地址
      - 微地址寄存器：接收微地址
    - 工作过程
      - 取微指令
        - 微程序入口自动送入CMAR
        - CM中读出微指令送入CMDR
      - 机器指令操作码字段通过微地址形成部件产生微程序的入口地址，并将其送入CMAR
      - 从CM中逐条取出对应的微指令并执行
      - 执行完后，继续从头循环往复
  - 微指令编码方式
    - 直接编码法
      - 优点：简单 直观不需要译码，速度快，操作并行性好
      - 缺点：对控制存储器容量有要求
    - 字段直接编码法
      - 优点：缩短指令的字长
      - 缺点：需要进行译码
      - ***互斥的指令放在相同段内，相容指令放在不同段内***
    - 字段间接编码法
      - 一个字段的微指令由另一个字段中的微指令解释
      - 优点：进一步缩短指令字长
      - 缺点：削弱了并行控制能力，该方法只是作为辅助手段
  - 微指令地址形成方式
    - 直接由微指令的下地址字段给出
    - 根据机器指令的操作码形成
  - 微指令格式
    - 水平型微指令
      - 优点：微程序短，执行速度快，并行能力强，灵活性高
      - 缺点：微指令长，编写微程序麻烦
    - 垂直型微指令
      - 优点：微指令短，简单，规整，便于编写微程序
      - 缺点：微程序长，执行速度慢，效率低下
    - 混合型指令　　结合二者优点 不长不短
  - 动态微程序设计和毫微程序设计
    - 动态微程序设计
      - 根据用户要求改变微程序
      - 采用EPROM
    - 毫微程序设计　　主存的每条指令都是放在控制存储器中的微程序解释执行的，通过控制线对硬件进行直接控制
- 硬布线和微程序控制器的特点
  - 硬布线
    - 优点：速度快
    - 缺点：不够灵活
    - RISC CPU
  - 微程序
    - 优点：规整 灵活 可维护性好
    - 缺点：速度慢
    - CISC CPU

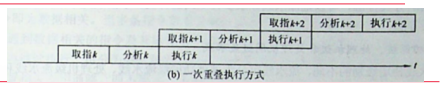
5.5指令流水线（上）

指令流水线的基本概念

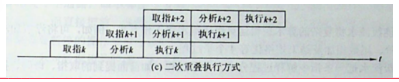
- 概念：把一个重复的过程分解成若干个子过程，每个子过程可以与其他子过程并行执行
- 优点：只需要增加少量的硬件就能把计算机的运算速度提高几倍
- 指令流水线的定义
  - 一条指令的执行可以分为多个阶段
    - 取指：根据PC从主存中取出指令送入IR
    - 分析：对指令操作码进行译码，按照寻址方式和地址段内容形成有效地址EA，并从有效地址中取出操作数
    - 执行：根据操作码字段，完成指令规定功能，将运算结果写到通用寄存器或者主存中
  - 多条指令的处理方式
    - 顺序执行方式
      - 一条接一条指令执行，传统冯诺依曼机顺序执行方式　　T=3nt
      - 优点：控制简单，硬件代价小
      - 缺点：执行指令速度慢，各功能部件的利用率很低
    - 一次重叠执行方式
      - 第k条指令的执行阶段和k+1条指令的取指阶段并行　　T=(1+2nt)
      - 优点：程序执行时间缩短了1/3，各功能部件利用率提高了
      - 缺点：硬件开销代价较大，控制过程也比顺序执行复杂
    - 二次重叠执行方式
      - k+1条指令提前到分析第k条指令的期间完成，而将分析第k+1条指令与执行第k条指令同时进行
      - 优点：时间更短了，利用率更高了
      - 缺点：硬件开销更大了，控制过程更复杂了
- 流水线的表示方法
  - 顺序执行方式



  - 一次重叠执行方式



  - 二次重叠执行方式



  - ***流水线最重要的是一定要流动起来，如果经常出现断流情况，那么就无法体现流水线的优越性***
- 流水线方式的特点
  - 一个任务分解成多个子任务
  - 每个功能部件后面都要有个锁存器，用于保存本流水段的结果
  - 流水线中的各功能段的时间应尽量相等，否则将会堵塞、断流
  - 流水线需要装入时间和排空时间
    - 装入时间：第一个任务进入流水线到输出流水线的时间
    - 排空时间：最后一个任务进入流水线到输出流水线的时间

流水线的分类

- 按照流水级别分类
  - 部件功能级流水线：将复杂的算术逻辑运算组成流水线的工作方式
  - 处理机级流水线：一条指令解释成多个子过程
  - 处理机间流水线：是一种宏流水，每个处理机专门完成一个任务，各个处理机得到的结果存放在与下一个处理机共享的存储器中
- 按照功能分类
  - 单功能流水线：完成单一功能
  - 多功能流水线：各个流水段之间通过组合可以实现多种功能
- 按照连接方式分类
  - 静态流水线：同一时间内，流水线的各段只能按照同一种功能的连接方式工作
  - 动态流水线：不同的段完成的运算可能不一样，可以提高效率，但是流水线控制变得很复杂
- 按照是否存在反馈信号分类
  - 线性流水线：不存在反馈回路
  - 非线性流水线：存在反馈回路，非常适合线性递归运算

5.5指令流水线（下）

影响流水线的因素

- 结构相关（资源冲突）
  - 概念：同一时刻争抢统一资源
  - 解决方法
    - 冲突指令之间，插入暂停周期
    - 单独设置数据存储器和指令存储器，但是增加了资源消耗
- 数据相关（数据冲突）
  - 概念：一个指令的执行必须要等待前一个指令的结果
  - 解决方法：　对数据相关的指令进行阻塞时钟周期
    - 硬件阻塞（stall）
    - 软件插入NOP指令
- 控制相关（控制冲突）
  - 概念：出现转移指令或者其他改变PC值得指令造成断流
  - 解决方法
    - 对转移指令进行分支预测
      - 简单（静态）预测
      - 动态预测　　准确率较高
    - 预取转移成功和不成功两个控制流方向上的目标指令
    - 加快和提前形成条件码
    - 提高转移方向的猜准率

流水线的性能指标

- 吞吐率
  - 单位时间内流水线完成的任务数量
  - TP=n/Tk　　n是任务数，Tk是处理完n个任务的时间
  - 当连续输入的任务n趋向于无穷时，此时为最大吞吐率
- 流水线的加速比
  - 不使用流水线与使用流水线的时间比值
  - $S=\frac{T_0}{T_k}$　　$S=\frac{kn\Delta t}{(k+n-1)\Delta t}=\frac{kn}{k+n-1}$
  - 连续任务数n趋向于无穷，此时为最大加速比
- 流水线效率
  - 完成n个任务使用的时空区的面积与n个任务所用的时间及k个流水段所围成的时空区总面积比
  - $E=\frac{n\text{个任务占用的时空区有效面积}}{n\text{个任务所用的时间与}k\text{个流水段所围成的时空区总面积}}=\frac{T_0}{kT_k}$
  - 连续输入任务n趋向于无穷时，最高效率为1

超标量流水线的基本概念

- 超标量流水线技术
  - 每个时钟周期内可以并发多条独立指令
  - 可以编译优化，把可并行执行的指令搭配起来，挖掘指令并行性
- 超流水线技术
  - 在一个时钟周期内再分段，在一个时钟周期内一个功能部件使用多次
  - 编译程序解决优化问题
- 超长指令字　　使用多个功能部件，利用编译程序挖掘出指令之间的并行性，然后将并行指令组成超长指令字

指令周期执行模板

取指令周期

- PC--->MAR　现行指令地址--->MAR
- 1-->R　命令存储器读
- M(MAR)-->MDR　现行指令从存储器中读至MDR
- MDR-->IR　现行指令-->IR
- OP(IR)-->CU　指令的操作码-->CU译码
- (PC)+1-->PC　形成下一条指令的地址

间址周期

- Ad（IR)-->MAR　将指令字中的地址码（形式地址）-->MAR
- 1-->R　命令存储器读
- M(MAR)-->MDR　将有效地址从存储器读至MDR

指令执行周期

非访存指令

- CLA　清ACC　0-->ACC
- COM　取反　$\overline{ACC} \rightarrow ACC$
- SHR　算数右移　L(ACC)-->R(ACC)　$ACC_0 \rightarrow ACC_n$
- CSL　循环左移　R(ACC)-->L(ACC)　$ACC_0 \rightarrow ACC_n$
- STP　停机指令　0--->G

访存指令

加法指令

- ADD　X　加法指令　Ad(IR)-->MAR
- 1->R
- M(MAR)-->MDR
- (ACC)+(MDR)-->ACC

存数指令

- STA　X　Ad（IR)-->MAR
- 1-->W
- ACC-->MDR
- MDR-->M(MAR)

取数指令

- LDA　X　Ad（IR)-->MAR
- 1-->R
- M(MAR)-->MDR
- MDR-->ACC

转移指令

- JMP　X　无条件转移　Ad（IR)-->PC
- BAN　X　条件转移（负则转）　$A_0 \cdot Ad(IR) + \overline{A_0} \cdot (PC) \rightarrow PC$

6.1 总线概述

- 总线基本概念
  - 总线的定义
    - 总线是一组能为多个部件分时共享的公共信息传送线路
    - 分时：同一时刻只允许有一个部件向总线发送信息
    - 共享：总线上可以挂接多个部件，各个部件之间互相交换的信息可以通过这组线路分时共享
  - 总线设备
    - 主设备：总线的主设备是指获得总线控制权的设备
    - 从设备：总线的从设备是指被主设备访问的设备，只能相应从主设备发来的各种总线命令
  - 总线特性
    - 机械特性：尺寸 形状
    - 电气特性：传输方向和有效的电平范围
    - 功能特性：每根传输线的功能
    - 时间特性：信号和时序的关系
  - 总线的猝发传输　一个总线周期内传输存储地址连续的多个数据字的总线传输方式，称为猝发传输
- 总线的分类
  - 片内总线：芯片内部的总线，是CPU芯片内部寄存器与寄存器之间，寄存器与ALU之间的公共连接线
  - 系统总线
    - 各个功能部件之间相互连接的总线
    - 数据总线
      - 传输各功能部件之间的数据信息
      - 双向传输总线
      - 位数与机器字长、存储字长有关
    - 地址总线
      - 数据总线上的源数据或目的数据所在的主存单元或I/O端口的地址
      - 单向传输总线
      - 地址总线的位数与主存地址空间的大小有关
    - 控制总线　传输控制信息　CPU送出的控制命令和主存返回CPU的反馈信号
  - 通信总线
    - 计算机系统之间或计算机系统与其他系统之间传送信息的总线，也被称为外部总线
    - 根据时序控制方式分类
      - 同步总线
      - 异步总线
    - 数据传输格式分类
      - 并行总线
      - 串行总线
- 系统总线的结构
  - 单总线结构
    - 将CPU 、主存、 I/O设备都挂载到一组总线上
    - 优点：结构简单 成本低 容易接入新设备
    - 缺点：带宽低 负载重 多个部件只能争用唯一的总线，并且不支持并发传送操作
  - 双总线结构
    - 一条是主存总线，用于CPU、主存和通道之间传送数据，另一条是I/O总线 用于在多个外部设备和通道之间传送数据
    - 优点：将低速I/O设备从单总线上分离出来，实现了存储总线和I/O总线分离
    - 缺点：需要增加通道等硬件设备
  - 三总线结构
    - 计算机系统各部件之间采用三条独立总线来构成信息通路（主存总线 I/O总线 直接内存访问（DMA）总线
    - 优点：提高了I/O设备的性能，使其更快的响应命令，提高系统的吞吐量
    - 缺点：系统工作效率较低
- 总线的性能指标
  - 总线的传输周期：一次总线操作所需要的时间（申请阶段，寻址阶段，传输阶段和结束阶段），总线传输周期由若干个总线时钟周期构成
  - 总线时钟周期：即机器的时钟周期
  - 总线的工作频率：总线周期的倒数
  - 总线的时钟频率：时钟周期的倒数
  - 总线带宽：单位时间内总线可以传输的数据位数
  - 总线复用：一种信号线在不同时间传输不同的信息，节约了空间和成本
  - 信号线数：地址总线、数据总线、控制总线，三种总线数的总和称为信号线数　总线带宽：总线本身的最大传输率，是衡量性能的重要指标

6.2总线仲裁

集中仲裁方式

链式查询方式
- 一根总线请求线 一根总线忙线 一根总线允许线
- 优点：优先级固定 结构简单 扩充容易
- 缺点：对于电路故障敏感 优先级不可变

越靠近总线控制器的部件优先级越高

计数器定时查询方式
- 使用计数器控制总线使用权
- 优点：设备优先级相等 优先次序可以改变
- 缺点:增加了控制线数 控制较为复杂

独立请求方式
- 每个设备均有请求线与总线允许线
- 优点：响应速度快 优先次序控制灵活
- 缺点：控制线数多 总线控制逻辑更复杂

对比项目 \ 仲裁方式	链式查询	计数器定时查询	独立请求
控制线数	3 （总线请求：1；总线忙：1；总线允许：1）	$\lceil \log_2 n \rceil + 2$ （总线请求：1；总线忙：1；总线允许：$\lceil \log_2 n \rceil$）	$2n+1$ （总线请求：n；总线允许：n；总线忙：1）
优点	优先级固定；结构简单；扩充容易	优先级灵活	响应速度快；优先级灵活
缺点	对电路故障敏感，优先级不灵活	控制线多；控制复杂	控制线多；控制复杂

分布仲裁方式

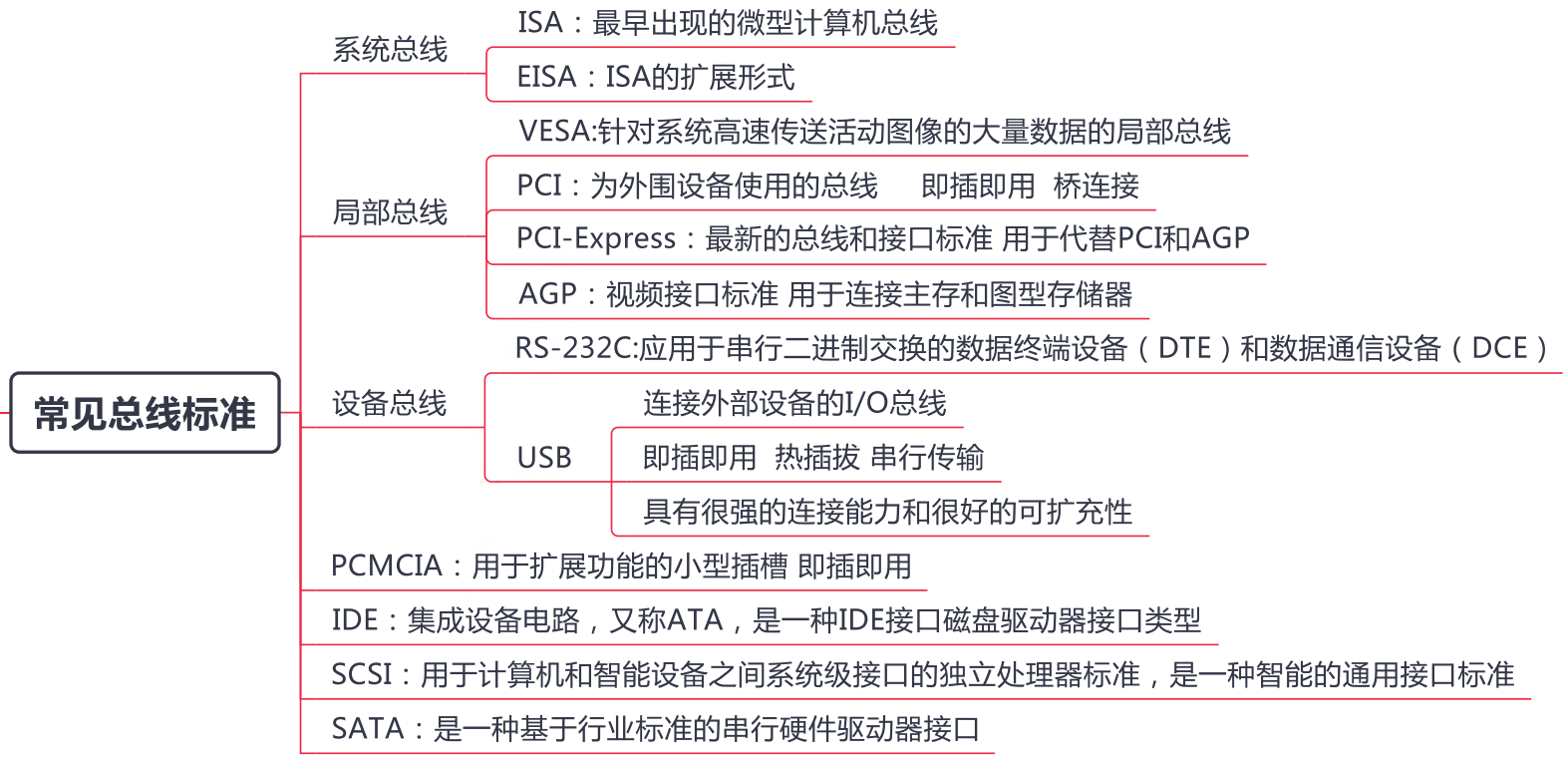
不需要中央仲裁器，每个潜在主模块，都有自己的仲裁号和仲裁器，然后进行仲裁号比较，获胜者仲裁号保留在仲裁总线上

6.3 总线操作和定时

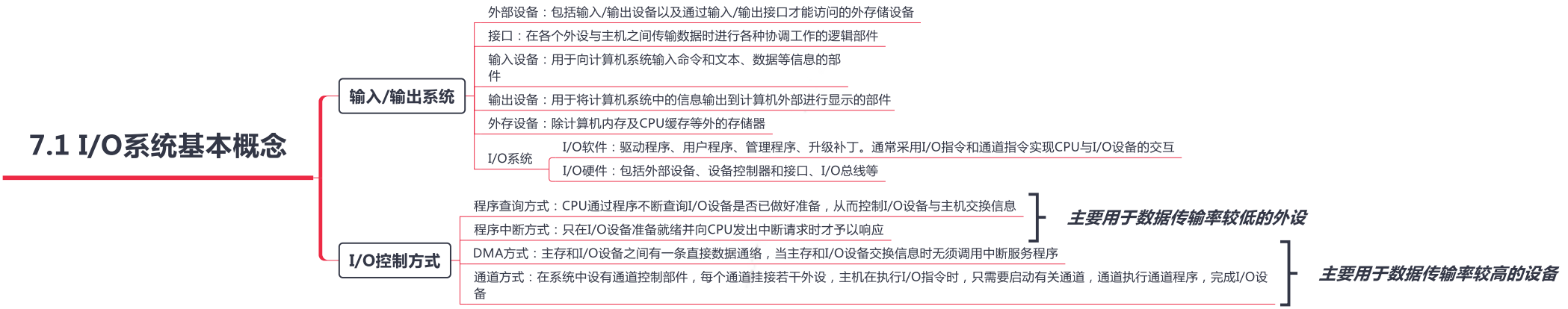
- **总线传输的4个阶段**
  - 申请分配阶段：设备提出申请
  - 寻址阶段：获得使用权的主设备，发出要访问模块的地址以及相关命令，启动本次传输的从模块
  - 传输阶段：主模块与从模块，进行双向或者单向数据传送
  - 结束阶段：主模块有关信息均从系统总线上撤除，让出总线使用权
- **同步定时方式**
  - 系统采用一个统一的时钟信号来协调发送和接收双方的传送定时关系
  - 优点：传送速度快 拥有较高的传输速率 总线控制逻辑简单
  - 缺点：主从设备属于强制性同步 不能及时进行数据通信的有效性检验 可靠性较差

  适用于总线长度较短以及总线所接部件的存取时间比较接近的系统
- **异步定时方式**
  - 没有统一的时钟，完全按照传送双方相互制约的握手信号实现定时控制
  - 优点：总线周期长度可变 可以保证两个速度相差较大部件或设备的信息传输
  - 缺点：控制相对复杂 并且速度相对较慢
  - 分类
    - 不互锁方式：主设备发送请求信号，不必等待从设备信号回答，一段时间便撤销请求信号
    - 半互锁方式：主设备发送请求信号，必须接到从设备应答信号才能撤销请求信号。
    - 全互锁方式：主设备发送请求信号，从设备应答，主设备撤销请求信号，从设备知晓主设备获得应答信号后撤销应答信号 套娃

6.4总线标准


常见总线标准
系统总线
ISA：最早出现的微型计算机总线
EISA：ISA的扩展形式
局部总线
VESA:针对系统高速传送活动图像的大量数据的局部总线
PCI：为外围设备使用的总线　即插即用 桥连接
PCI-Express：最新的总线和接口标准 用于代替PCI和AGP
AGP：视频接口标准 用于连接主存和图型存储器
设备总线
RS-232C:应用于串行二进制交换的数据终端设备（DTE）和数据通信设备（DCE）
USB
连接外部设备的I/O总线
即插即用 热插拔 串行传输
具有很强的连接能力和很好的可扩充性
PCMCIA：用于扩展功能的小型插槽 即插即用
IDE：集成设备电路，又称ATA，是一种IDE接口磁盘驱动器接口类型
SCSI：用于计算机和智能设备之间系统级接口的独立处理器标准，是一种智能的通用接口标准
SATA：是一种基于行业标准的串行硬件驱动器接口

7.1 I/O系统基本概念


输入/输出系统
外部设备：包括输入/输出设备以及通过输入/输出接口才能访问的外存储设备
接口：在各个外设与主机之间传输数据时进行各种协调工作的逻辑部件
输入设备：用于向计算机系统输入命令和文本、数据等信息的部件
输出设备：用于将计算机系统中的信息输出到计算机外部进行显示的部件
外存设备：除计算机内存及CPU缓存等外的存储器
I/O系统
I/O软件：驱动程序、用户程序、管理程序、升级补丁。通常采用I/O指令和通道指令实现CPU与I/O设备的交互
I/O硬件：包括外部设备、设备控制器和接口、I/O总线等
I/O控制方式
程序查询方式：CPU通过程序不断查询I/O设备是否已做好准备，从而控制I/O设备与主机交换信息
程序中断方式：只在I/O设备准备就绪并向CPU发出中断请求时才予以响应
主要用于数据传输率较低的外设
DMA方式：主存和I/O设备之间有一条直接数据通络，当主存和I/O设备交换信息时无须调用中断服务程序
通道方式：在系统中设有通道控制部件，每个通道挂接若干外设，主机在执行I/O指令时，只需要启动有关通道，通道执行通道程序，完成I/O设备
主要用于数据传输率较高的设备

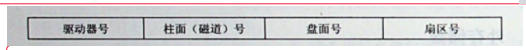
7.2 外部设备（上）


输入设备
键盘
运作方式：1、查出按下的是哪个键 2、将该键翻译成能被主机接收的键码（如ASCII码）3、将编码送给主机
鼠标
运行方式：鼠标在平面上移动，底部传感器将移动距离与方向检测出来，从而控制光标的移动
常见的鼠标有 机械式与光电式
输出设备
显示器
显示器件分类
阴极射线管(CRT)显示器
组成：电子枪，偏转线圈，萌罩、高压石墨电极，荧光粉涂层和玻璃外壳
优点：可视角度大，无坏点，色彩还原度高，色度均匀，可调节的多分辨率模式，响应时间短
分类
字符显示器
显示字符的方法以点阵为基础
图形显示器
将所显示图形的一组坐标点和绘图命令组成显示文件存放在缓冲存储器中的显示文件传送给矢量产生器，产生相应的模拟电压，直接控制电子束在屏幕上移动
优点：分辨率高且显示的曲线平滑
缺点：显示复杂图形时，会有闪烁感
液晶显示器（LCD）
原理：利用液晶的电光效应，由图像信号电压直接控制薄膜晶体管，再简介控制液晶分子的光学特性来实现图像的显示
特点：体积小 重量轻 省电 无辐射 绿色环保 画面柔和 不伤眼
发光二极管（LED）显示器
原理：通过控制半导体发光二极管来显示文字、图形、图像等各种信息
特点：亮度 功耗 可视角度和刷新速率等方面优于LCD
按照显示信息分类
字符显示器
图形显示器
图像显示器
主要参数
屏幕大小：对角线长度
分辨率：所能显示的像素个数，宽与高的乘积
灰度级：黑白显示器中所显示的像素点的亮暗差别，彩色显示器中颜色的不同 灰度级越高图形越清晰越逼真
刷新：光点只能保持极短的时间就会消失，必须在其消失前重新扫描一遍
刷新频率：单位时间内扫描整个屏幕内容的次数，刷新频率大于30hz时人眼无法分辨出刷新，通常显示器的刷新频率为60~120HZ
显示存储器(VRAM）：也称为刷新存储器，将一帧的图像存储在刷新存储器中，从而提高刷新图像的信号
分辨率越高，灰度级越多，刷新存储器容量越大
VRAM容量 = 分辨率X灰度级位数
VRAM带宽 = 分辨率X灰度级位数X帧频
打印机
针式打印机
原理：主机发出打印命令，经过接口、检测和控制电路，间歇驱动纵向送纸和打印头横向移动，同时驱动打印机间歇冲击色带打印内容
优点：擅长多层复习打印 实现各种票据和蜡纸等打印 工作原理简单 造价低廉 耗材便宜
缺点：打印分辨率和打印速度不够高
喷墨式打印机
原理：带电的喷墨雾点经过电极偏转后，直接再纸上形成所需要的字形
优点：打印噪声小 实现高质量彩色打印 打印速度比针式打印机快
缺点：防水性 打印成本高 需要专用打印纸
激光打印机
原理：计算机输出二进制信息，经过调制后的激光束扫描，在感光鼓上形成潜像，经过显影、转印和定影，在纸上得到所需的字符或图像
优点：打印质量高 速度快 噪音小 处理能力强
缺点：耗材多 价格贵 不能复写打印多份 对纸张要求高

7.2 外部设备（下）

磁表面存储器

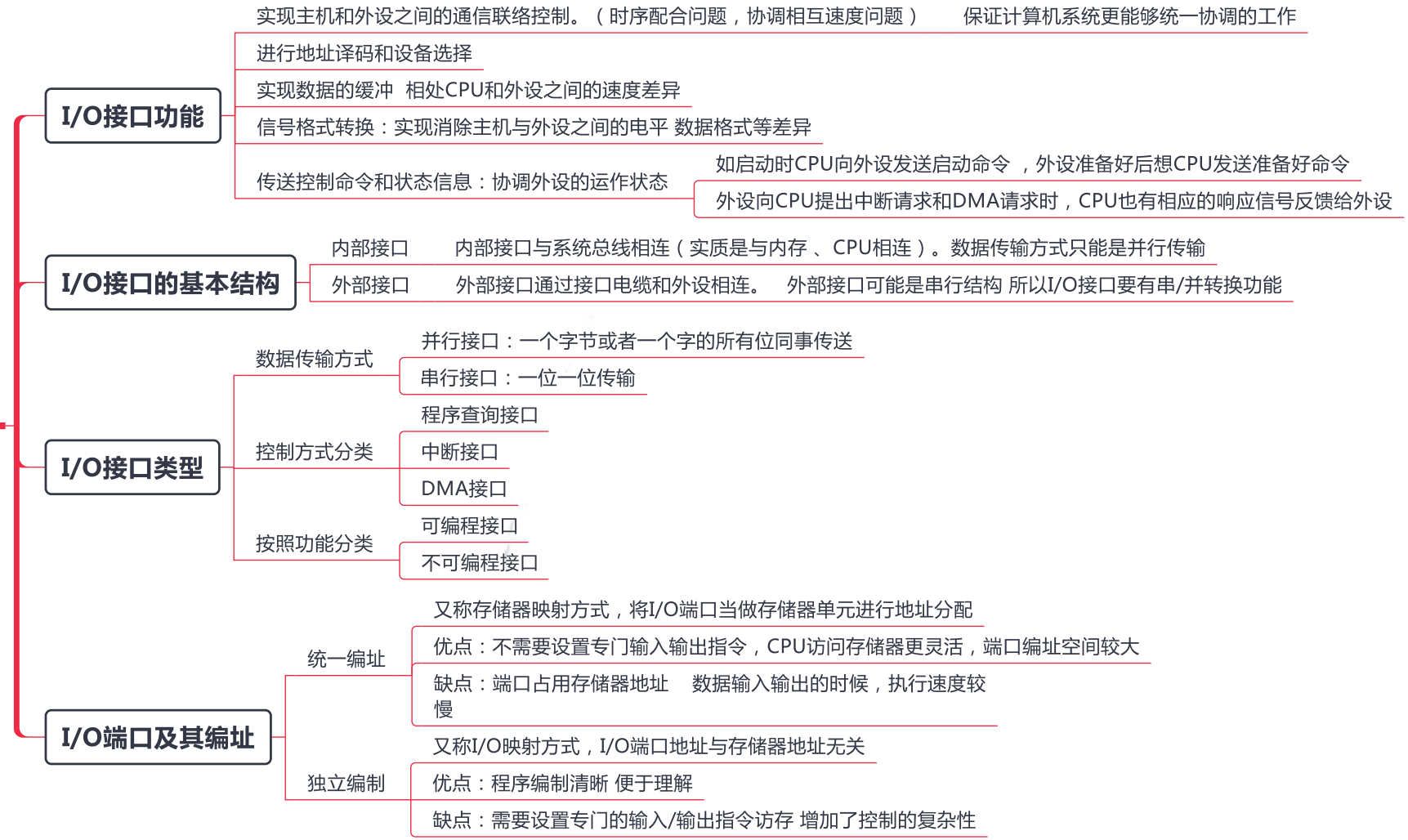
- 优点： 存储容量大 价格低 非破坏性读出 记录信息可以长久保存
- 缺点：存取速度慢 机械机构复杂 对工作环境要求高
- 磁盘存储器
  - 组成
    - 存储区域
      - 磁头：用于读取/写入盘片上的记录面信息，一个记录面对应一个磁头
      - 柱面数：表示硬盘每个盘片上有多少磁道
      - 扇片数：表示每条磁道上有多少个扇区
    - 硬盘存储器
      - 磁盘驱动器：核心部件是磁头组件和盘片组件
      - 磁盘控制器：硬盘存储器和主机的接口，IDE SCSI SATA等
  - 磁记录原理
    - 原理：磁头和磁性记录介质相对运动，通过电转磁完成读/写操作
    - 编码方法：按照某种方案把一连串二进制信息变换成存储介质磁层中一个磁化翻转状态的序列，并使读/写控制电路容易、可靠的实现变换
    - 磁记录方式：通常采用调频制（FM）和改进型调频制（MFM）的记录方式
  - 磁盘的性能指标
    - 磁盘的容量：存储字节的数量　　格式化容量：按照某种特定记录方式可以记录信息的总量　　非格式化容量：可以利用的磁化单元总数
    - 记录密度：盘片单位面积上记录得二进制信息量
      - 道密度：沿磁盘半径方向单位长度上的磁道数
      - 位密度：磁道单位长度上能记录得二进制代码位数
      - 面密度：位密度和道密度的乘积
    - 平均存取时间：寻道时间（磁头移动到目的磁道的时间）+旋转延迟时间（磁头定位到所在扇区的时间）+传输时间（传输数据所花费的时间）
    - 数据传输率：磁盘存储器在单位时间内向主机传送数据的字节数。 $D_r = rN$
  - 磁盘地址　　组成：

    - 按照二进制对其对应个数进行编码
  - 硬盘工作过程
    - 寻址、读盘、写盘
    - 第一步是取控制字 第二步是执行控制字
    - 读写操作是串行的，不能既读又写
- 磁盘阵列
  - RAID 是指将多个独立的物理磁盘组成一个独立的逻辑盘，数据在多个物理盘上分割交叉存储，并行访问，具有更好的存储性能、可靠性和安全性
  - RAID分级
    - RAID0：无冗余和无校验的磁盘阵列
    - RAID1：镜像磁盘阵列
    - RAID2：采用纠错的海明码的磁盘阵列
    - RAID3：位交叉奇偶校验的磁盘阵列
    - RAID4：块交叉奇偶校验的磁盘真理
    - RAID5：无独立校验的就校验磁盘阵列

    提高了传输率 数据吞吐量 提高了安全性 提高了容错能力
- 光盘存储器
  - 原理：利用光学原理读/写信息的存储装置，采用聚集激光束对盘式介质以非接触方式记录信息
  - 特点：具有高存储密度 携带方便 成本低 容量大 存储期限长 容易保存
  - 分类
    - CD-ROM：只读型光盘
    - CD-R：只能写入一次信息 之后不可修改
    - CD-RW： 可读可写光盘 可以重复读写
    - DVD-ROM：高容量CD-ROM DVD表示通用数字化多功能光盘
- 固态硬盘：常用于高档笔记本电脑

7.3 I/O接口


I/O接口功能
实现主机和外设之间的通信联络控制。（时序配合问题，协调相互速度问题）　保证计算机系统更能够统一协调的工作
进行地址译码和设备选择
实现数据的缓冲　相处CPU和外设之间的速度差异
信号格式转换：实现消除主机与外设之间的电平 数据格式等差异
传送控制命令和状态信息：协调外设的运作状态
如启动时CPU向外设发送启动命令 ，外设准备好后想CPU发送准备好命令
外设向CPU提出中断请求和DMA请求时，CPU也有相应的响应信号反馈给外设
I/O接口的基本结构
内部接口　内部接口与系统总线相连（实质是与内存 、CPU相连）。数据传输方式只能是并行传输
外部接口　外部接口通过接口电缆和外设相连。　外部接口可能是串行结构 所以I/O接口要有串/并转换功能
I/O接口类型
数据传输方式
并行接口：一个字节或者一个字的所有位同事传送
串行接口：一位一位传输
控制方式分类
程序查询接口
中断接口
DMA接口
按照功能分类
可编程接口
不可编程接口
I/O端口及其编址
统一编址
又称存储器映射方式，将I/O端口当做存储器单元进行地址分配
优点：不需要设置专门输入输出指令，CPU访问存储器更灵活，端口编址空间较大
缺点：端口占用存储器地址　数据输入输出的时候，执行速度较慢
独立编制
又称I/O映射方式，I/O端口地址与存储器地址无关
优点：程序编制清晰 便于理解
缺点：需要设置专门的输入/输出指令访存 增加了控制的复杂性

7.4 IO方式（上）

- **程序查询方式**
  - 原理：信息交换完全交给主机执行程序实现，主机对设备的状态进行询问，然后根据结果决定下一步是传送数据还是等待
  - 优点：设计简单且设备量小
  - 缺点：CPU需要花费大量时间进行查询和等待　一段时间内只能和一台外设交互信息 效率低　CPU存在原地踏步现象
- **程序中断方式**
  - 中断的基本概念
    - 当计算机出现异常情况或者特殊请求，CPU暂时中止当前程序，转去处理异常或者特殊情况
    - 作用
      - 实现CPU与IO并行工作
      - 处理硬件故障和软件错误
      - 实现人机交互，用户干预机器需要用到中断系统
      - 实现多道程序、分时操作，多道程序的切换需借助于中断系统
      - 实时处理需要借助中断系统来实现快速响应
      - 实现应用程序和操作系统（管态程序）的切换，称为"软中断"
      - 多处理器系统中各处理器之间的信息交流和任务切换
  - 工作流程
    - 中断请求
      - 内/外中断
        - 内中断：内中断主要是指在处理器和内存内部产生的中断，包括程序运算引起的各种错误，如地址非法、校验错等
        - 外中断：外中断是指来自处理器和内存以外的部件引起的中断，包括I/O设备发出的I/O中断等
      - 硬件中断与软件中断
        - 硬件中断：通过外部的硬件产生的中断，硬件中断属于外中断
        - 软件中断：通过某条指令产生的中断，这种中断是可以编程实现的，软件中断是内中断
      - 非屏蔽与可屏蔽中断
        - 非屏蔽中断：非屏蔽中断是一种硬件中断，不受中断标志位IF的影响，即使在关中断（IF = 0）的情况下也会被响应。
        - 可屏蔽中断：可屏蔽中断也是一种硬件中断，受中断标志位IF的影响，在关中断情况下不接受中断请求
    - 中断判优
      - 通过中断判优逻辑确定响应哪个中断源的请求
      - 硬件实现：硬件实现是通过硬件排队器实现的，它既可以设置在CPU中，又可以分散在各个中断源中
      - 软件实现：软件实现是通过查询程序实现的
      - 一般逻辑
        - 硬件故障中断属于最高级
        - 软件中断
        - 非屏蔽中断优于可屏蔽中断
        - DMA 请求优于I/O设备传送的中断请求
        - 高速设备优于低速设备
        - 输入设备优于输出设备
        - 实时设备优于普通设备
    - CPU响应中断条件
      - 中断源有中断请求
      - CPU允许中断及开中断
      - 一条指令执行完毕，且没有更紧迫的任务
    - 中断隐指令
      - CPU响应中断后，经过某些操作，转去执行中断服务程序
      - 完成操作
        - 关中断：保证被中断的程序在中断服务程序执行完毕后能接着正确地执行
        - 保存断点：将原来的PC内容保存
        - 引出中断服务程序：取出中断服务程序的入口地址并传送给程序计数器(PC)
    - 中断向量：中断服务程序的入口地址　该方法叫做中断向量法
    - 中断处理过程
      - 关中断
      - 保存断点
      - 引出中断服务程序
      - 保存现场和屏蔽字
      - 开中断
      - 执行中断服务程序
      - 关中断
      - 恢复现场和屏蔽字
      - 开中断、中断返回

7.4 IO方式（下）

- 多重中断和中断屏蔽技术
  - 处理中断的时候又来了中断　套娃
  - 多重中断功能具备的条件
    - 中断服务程序中设置开指令
    - 优先级别高的中断源有权中断优先级别低的中断源
- DMA方式
  - 概述
    - DMA方式是一种完全由硬件进行成组信息传送的控制方式，DMA方式在外设与内存之间开辟一条“直接数据通道”
    - 适用于磁盘机 磁带机等高速设备
    - 硬件开销大
  - 特点
    - 它使主存与CPU的固定联系脱钩，主存既可被CPU访问，又可被外设访问
    - 在数据块传送时，主存地址的确定、传送数据的计数等都由硬件电路直接实现
    - 主存中要开辟专用缓冲区，及时供给和接收外设的数据
    - DMA传送速度快，CPU和外设并行工作，提高了系统效率
    - DMA在传送开始前要通过程序进行预处理，结束后要通过中断方式进行后处理
  - 组成
    - 主存地址计数器：存放要交换数据的主存地址
    - 传送长度计数器：记录传送数据的长度，计数溢出时，数据即传送完毕，自动发中断请求信号
    - 数据缓冲寄存器：暂存每次传送的数据
    - DMA请求触发器：I/O发出控制信号，使得DMA请求触发置位
    - “控制/状态”逻辑：由控制和时序电路及状态标志组成，用于指定传送方向，修改传送参数，并对DMA请求信号和CPU响应信号进行协调和同步。
    - 中断机构：当一个数据块传送完毕后触发中断机构，向CPU提出中断请求
  - 传送方式
    - 停止CPU访问主存：CPU放弃地址线、数据线和有关控制线的使用权，DMA接口获得总线控制权
    - DMA与CPU交替访存：这种方式适用于CPU的工作周期比主存存取周期长的情况
    - 周期挪用（或周期窃取）
      - CPU不在访存，I/O的访存请求与CPU未发 生冲突
      - CPU正在访存，此时必须待存取周期结束后，CPU再将总线占有权让出
      - I/O和CPU同时请求访存，出现访存冲突，此时CPU要暂时放弃总线占有权，由I/O设备挪用一个或几个存取周期
  - 传送过程
    - 预处理：由CPU完成一些必要的准备工作（寄存器置初值、设置传送方向、启动该设备）
    - 数据传送：DMA的数据传输可以以单字节（或字）为基本单位，也可以以数据块为基本单位 ，数据传送阶段完全由DMA （硬件）控制
    - 后处理：DMA控制器向CPU发送中断请求，CPU执行中断服务程序做DMA结束处理
- DMA方式与中断方式的区别
  - 中断方式是程序的切换，需要保护和恢复现场　DMA方式除了预处理和后处理，其他时候不占用CPU的任何资源
  - 中断请求的响应只能发生在每条指令执行完毕时　DMA 请求的响应可以发生在每个机器周期结束时
  - 中断传送过程需要CPU的干预　DMA传送过程不需要CPU的干预，适合于高速外设的成组数据传送
  - DMA请求的优先级高于中断请求
  - 中断方式具有对异常事件的处理能力　中断方式具有对异常事件的处理能力
  - 中断方式具有对异常事件的处理能力　DMA方式靠硬件传送